```
# -*- coding: utf-8 -*-
"""Kassa-bot v4 — baza (eski bazaga mos, ma'lumot yo'qolmaydi), parsing, hisob-ki
Telegramga bog'liq emas, shuning uchun alohida sinaladi."""
import os
import re
import sqlite3
import datetime as dt
from collections import OrderedDict
from zoneinfo import ZoneInfo

TZ = ZoneInfo(os.getenv("TZ_NAME", "Asia/Tashkent"))
DB_PATH = os.getenv("DB_PATH", "kassa.db")
ANON = 1087968824                       # "anonim admin" akkaunti
DAY_CUTOFF_HOUR = int(os.getenv("DAY_CUTOFF_HOUR", "0"))
CHANGE_DAY = "2026-10-12"               # yangi foizlar shu haftaning dushanbasida


from decimal import Decimal, ROUND_HALF_UP


def r2(v):
    """Pul yaxlitlash: 0.915 -> 0.92 (oddiy maktab usuli, float xatosisiz)."""
    return float(Decimal(repr(round(float(v), 8))).quantize(Decimal("0.01"), roun


def today():
    return dt.datetime.now(TZ).date()


def iso(d):
    return d.isoformat()


def week_start(d=None):
    d = d or today()
    return d - dt.timedelta(days=d.weekday())


def post_day(msg_dt):
    return (msg_dt.astimezone(TZ) - dt.timedelta(hours=DAY_CUTOFF_HOUR)).date()


# ──────────────────────────── baza ────────────────────────────
```

```
SCHEMA = """
CREATE TABLE IF NOT EXISTS meta(k TEXT PRIMARY KEY, v TEXT);
CREATE TABLE IF NOT EXISTS workers(id INTEGER PRIMARY KEY AUTOINCREMENT, name TEX
  is_self INTEGER NOT NULL DEFAULT 0, active INTEGER NOT NULL DEFAULT 1);
CREATE TABLE IF NOT EXISTS pct(worker_id INTEGER NOT NULL, from_day TEXT NOT NULL
  PRIMARY KEY(worker_id, from_day));
CREATE TABLE IF NOT EXISTS users(tg_id INTEGER PRIMARY KEY, name TEXT, role TEXT
CREATE TABLE IF NOT EXISTS chats(chat_id INTEGER PRIMARY KEY, title TEXT, watch I
CREATE TABLE IF NOT EXISTS prices(id INTEGER PRIMARY KEY AUTOINCREMENT, lo INTEGE
CREATE TABLE IF NOT EXISTS reports(id INTEGER PRIMARY KEY AUTOINCREMENT, worker_i
  sender_id INTEGER, raw TEXT, status TEXT NOT NULL DEFAULT 'pending', created TE
CREATE TABLE IF NOT EXISTS checks(id INTEGER PRIMARY KEY AUTOINCREMENT, report_id
  amount INTEGER NOT NULL, usd REAL, skip INTEGER NOT NULL DEFAULT 0);
CREATE TABLE IF NOT EXISTS adj(id INTEGER PRIMARY KEY AUTOINCREMENT, worker_id IN
  day TEXT NOT NULL, raw TEXT, chat_id INTEGER, msg_id INTEGER, cancelled INTEGER
CREATE TABLE IF NOT EXISTS advances(id INTEGER PRIMARY KEY AUTOINCREMENT, worker_
  day TEXT NOT NULL, note TEXT);
CREATE TABLE IF NOT EXISTS transfers(id INTEGER PRIMARY KEY AUTOINCREMENT, day TE
  to_id INTEGER NOT NULL, pct REAL NOT NULL, cancelled INTEGER NOT NULL DEFAULT 0
CREATE TABLE IF NOT EXISTS shares(id INTEGER PRIMARY KEY AUTOINCREMENT, to_id INT
  pct REAL NOT NULL, base TEXT NOT NULL DEFAULT 'zp', mode TEXT NOT NULL DEFAULT
  from_day TEXT NOT NULL DEFAULT '2000-01-01', active INTEGER NOT NULL DEFAULT 1)
CREATE INDEX IF NOT EXISTS idx_checks_report ON checks(report_id);
CREATE INDEX IF NOT EXISTS idx_checks_rd ON checks(rd);
CREATE INDEX IF NOT EXISTS idx_reports_worker ON reports(worker_id, status);
CREATE INDEX IF NOT EXISTS idx_adj_worker ON adj(worker_id, day);
"""

SEED_PRICES = [
    (400, 700, 4), (750, 1150, 5), (1400, 1400, 7), (1650, 2250, 7),
    (2300, 2800, 10), (3300, 3700, 13), (4000, 4000, 17), (6000, 6000, 25),
    (7000, 7000, 30), (9000, 12200, 100), (19200, 19200, 200), (49200, 49200, 500
]
# (ism, o'zi?, [(qaysi kundan, foiz)]) — eski foizlar tarix uchun; yangilari migr
SEED_WORKERS = [
    ("Cashliy", 1, 50), ("Bayzed", 0, 36), ("Cred", 0, 30), ("Baxa", 0, 30), ("La
]
NEW_PERCENTS = {"Cashliy": 50, "Bayzed": 41, "Cred": 36, "Baxa": 30}

_db = None


def _cols(c, t):
    return [r[1] for r in c.execute(f"PRAGMA table_info({t})")]
```

```
def _migrate(c):
    """Faqat QO'SHADI, hech narsani o'chirmaydi (haftalik hisoblar saqlanadi)."""
    if "skip" not in _cols(c, "checks"):
        c.execute("ALTER TABLE checks ADD COLUMN skip INTEGER NOT NULL DEFAULT 0"
    if "pay" not in _cols(c, "checks"):
        c.execute("ALTER TABLE checks ADD COLUMN pay INTEGER")
    if "sender_name" not in _cols(c, "reports"):
        c.execute("ALTER TABLE reports ADD COLUMN sender_name TEXT")
    if "kind" not in _cols(c, "workers"):
        c.execute("ALTER TABLE workers ADD COLUMN kind TEXT NOT NULL DEFAULT 'wor
    if not c.execute("select 1 from meta where k='v4'").fetchone():
        c.execute("update workers set kind='self' where is_self=1")
        c.execute("update workers set kind='pool' where lower(name)='lazarus' and
        if not c.execute("select 1 from workers where kind='partner'").fetchone()
            ex = c.execute("select id from workers where lower(name)='artur'").fe
            if ex:
                c.execute("update workers set kind='partner', is_self=0 where id=
                aid = ex[0]
            else:
                aid = c.execute("insert into workers(name, is_self, kind) values(
            if not c.execute("select 1 from pct where worker_id=?", (aid,)).fetch
                c.execute("insert into pct values(?,?,?)", (aid, "2000-01-01", 50
        for name, p in NEW_PERCENTS.items():
            w = c.execute("select id from workers where lower(name)=lower(?)", (n
            if w:
                c.execute("delete from pct where worker_id=? and from_day>=?", (w
                c.execute("insert into pct values(?,?,?)", (w[0], CHANGE_DAY, p))
        bz = c.execute("select id from workers where lower(name)='bayzed'").fetch
        bx = c.execute("select id from workers where lower(name)='baxa'").fetchon
        if bz and bx and not c.execute("select 1 from shares").fetchone():
            c.execute("insert into shares(to_id, from_id, pct, base, mode, from_d
                      (bz[0], bx[0], CHANGE_DAY))
        for a in c.execute("select worker_id, usd, day from advances").fetchall()
            c.execute("insert into adj(worker_id, kind, usd, day, raw) values(?,?
                      (a[0], "adv", a[1], a[2], "аванс (старый)"))
        c.execute("insert into meta values('v4','1')")


def db():
    global _db
    if _db is None:
        _db = sqlite3.connect(DB_PATH, check_same_thread=False)
        _db.row_factory = sqlite3.Row
        try:
            _db.execute("PRAGMA journal_mode=WAL")
            _db.execute("PRAGMA synchronous=NORMAL")
```

```
        except Exception:
            pass
        _db.executescript(SCHEMA)
        if not _db.execute("select 1 from workers").fetchone():
            for name, selfw, p in SEED_WORKERS:
                cur = _db.execute("insert into workers(name,is_self) values(?,?)"
                _db.execute("insert into pct values(?,?,?)", (cur.lastrowid, "200
        if not _db.execute("select 1 from prices").fetchone():
            _db.executemany("insert into prices(lo,hi,usd) values(?,?,?)", SEED_P
        _migrate(_db)
        _db.commit()
    return _db


def q(sql, args=()):
    return db().execute(sql, args).fetchall()


def q1(sql, args=()):
    return db().execute(sql, args).fetchone()


def x(sql, args=()):
    cur = db().execute(sql, args)
    db().commit()
    return cur


# ─────────────────────────── ishchilar ──────────────────────────
PAYABLE = ("worker", "self")           # oylik oladiganlar


def workers(active_only=True, kinds=None):
    sql = "select * from workers where 1=1"
    if active_only:
        sql += " and active=1"
    if kinds:
        sql += " and kind in (" + ",".join("'%s'" % k for k in kinds) + ")"
    return q(sql + " order by (kind='self') desc, (kind='partner') asc, (kind='po


def worker(wid):
    return q1("select * from workers where id=?", (wid,))


def self_worker():
```

```
    return q1("select * from workers where kind='self'")


def partner_worker():
    return q1("select * from workers where kind='partner'")


def percent_on(wid, day):
    d = day if isinstance(day, str) else iso(day)
    r = q1("select percent from pct where worker_id=? and from_day<=? order by fr
    return r["percent"] if r else 0.0


def add_worker(name, percent, from_day="2000-01-01"):
    cur = x("insert into workers(name) values(?)", (name.strip(),))
    x("insert into pct values(?,?,?)", (cur.lastrowid, from_day, percent))
    return cur.lastrowid


def set_percent(wid, percent, from_day):
    x("insert or replace into pct values(?,?,?)", (wid, from_day, percent))


def rename_worker(wid, name):
    x("update workers set name=? where id=?", (name.strip(), wid))


def set_active(wid, v):
    x("update workers set active=? where id=?", (v, wid))


def name_taken(name):
    return q1("select 1 from workers where lower(name)=lower(?)", (name.strip(),)


def worker_by_name(text, kinds=("worker", "self", "pool")):
    for w in sorted(workers(kinds=kinds), key=lambda w: -len(w["name"])):
        if re.search(r"(?<!\w)" + re.escape(w["name"]) + r"(?!\w)", text, re.I):
            return w["id"]
    return None


# ──────────────────────── foydalanuvchilar / chatlar ────────────────────────
def user(tg_id):
    return q1("select * from users where tg_id=?", (tg_id,))
```

```
def upsert_user(tg_id, name, role, worker_id):
    x("insert into users(tg_id,name,role,worker_id) values(?,?,?,?) "
      "on conflict(tg_id) do update set name=excluded.name, role=excluded.role, w
      (tg_id, name, role, worker_id))


def touch_user(tg_id, name):
    u = user(tg_id)
    if u and name and u["name"] != name:
        x("update users set name=? where tg_id=?", (name, tg_id))


def admins():
    return [r["tg_id"] for r in q("select tg_id from users where role='admin'")]


def owners():
    return [r["tg_id"] for r in q("select tg_id from users where role='owner'")]


def users_of_worker(wid):
    return q("select * from users where worker_id=?", (wid,))


def all_users():
    return q("select * from users order by role, tg_id")


def del_user(tg_id):
    x("delete from users where tg_id=?", (tg_id,))


def set_user_worker(tg_id, wid):
    x("update users set worker_id=? where tg_id=?", (wid, tg_id))


def set_role(tg_id, role):
    x("update users set role=? where tg_id=?", (role, tg_id))


def chat(cid):
    return q1("select * from chats where chat_id=?", (cid,))


def all_chats():
```

```
    return q("select * from chats order by title")


def upsert_chat(cid, title):
    x("insert into chats(chat_id,title) values(?,?) on conflict(chat_id) do updat


def set_watch(cid, v):
    x("update chats set watch=? where chat_id=?", (v, cid))


# ─────────────────────────── narx-ro'yxati ───────────────────────────
def price_for(amount):
    r = q1("select usd from prices where lo<=? and hi>=? order by (hi-lo) asc lim
    return r["usd"] if r else None


def price_row(pid):
    return q1("select * from prices where id=?", (pid,))


def list_prices():
    return q("select * from prices order by lo")


def refresh_prices():
    for c in q("select id, amount from checks where usd is null"):
        p = price_for(c["amount"])
        if p is not None:
            x("update checks set usd=? where id=?", (p, c["id"]))


def reprice_week():
    """Narx o'zgarsa shu haftaning dushanbasidan boshlab cheklar qayta narxlanadi
    mon = iso(week_start())
    for c in q("select id, amount from checks where rd>=?", (mon,)):
        p = price_for(c["amount"])
        if p is not None:
            db().execute("update checks set usd=? where id=?", (p, c["id"]))
    db().commit()


def add_price(lo, hi, usd):
    x("insert into prices(lo,hi,usd) values(?,?,?)", (min(lo, hi), max(lo, hi), u
    refresh_prices()
    reprice_week()
```

```
def update_price(pid, lo, hi, usd):
    x("update prices set lo=?, hi=?, usd=? where id=?", (min(lo, hi), max(lo, hi)
    refresh_prices()
    reprice_week()


def del_price(pid):
    x("delete from prices where id=?", (pid,))
    reprice_week()


# ─────────────────────────── parsing ─────────────────────────────
RD_FIND = re.compile(r"(?<![A-Za-zA-Яа-яЁё])(?:rd|рд)\W{0,3}(\d{1,2})[./-](\d{1,2
RD_WORD = re.compile(r"(?<![A-Za-zA-Яа-яЁё])(?:rd|рд)(?![A-Za-zA-Яа-яЁё])", re.I)
DATE_RE = re.compile(r"(?<!\d)\d{1,2}[./]\d{1,2}(?:[./]\d{2,4})?(?!\d)")
DATE_G = re.compile(r"(?<!\d)(\d{1,2})[./](\d{1,2})(?:[./](\d{2,4}))?(?!\d)")
TIME_RE = re.compile(r"\d{1,2}:\d{2}")
# 1950 | 1.950 | 1,950 | 19 200 (bo'shliq faqat 1-2 xonali boshi bilan: "950 600"
NUM_RE = re.compile(r"(?<!\d)(\d{1,3}(?:[.,]\d{3})+|\d{1,2}(?: \d{3})+|\d{3,6})(?
TOTAL_RE = re.compile(r"итого|всего|jami|umumiy|total|касса|kassa|сумма|зп\b|зарп


def mk_date(d, m, y=None, ref=None):
    ref = ref or today()
    try:
        if y:
            y = int(y)
            y = 2000 + y if y < 100 else y
            return dt.date(y, int(m), int(d))
        r = dt.date(ref.year, int(m), int(d))
        if r > ref + dt.timedelta(days=45):
            r = dt.date(ref.year - 1, int(m), int(d))
        return r
    except ValueError:
        return None


def find_rd(line, ref=None):
    m = RD_FIND.search(line)
    return mk_date(m.group(1), m.group(2), m.group(3), ref) if m else None


def parse_dates(text, ref=None):
    out = []
```

```
    for m in DATE_G.finditer(text):
        d = mk_date(m.group(1), m.group(2), m.group(3), ref)
        if d:
            out.append(d)
    return out


def parse_date_text(text, ref=None):
    ds = parse_dates(text, ref)
    return ds[0] if ds else None


def has_rd(text):
    return bool(RD_FIND.search(text))


def is_report_like(text):
    """Guruhdagi xabar faqat raqam/sana/Rd/ajratgichdan iborat bo'lsa hisobot. Bo
    t = RD_WORD.sub(" ", text.strip())
    return bool(t.strip()) and re.fullmatch(r"[\d\s,.;+=:/\-—]+", t) is not None


def line_amounts(line):
    if TOTAL_RE.search(line):
        return []
    body = line.rsplit("=", 1)[0] if "=" in line else line
    body = TIME_RE.sub(" ", DATE_RE.sub(" ", RD_FIND.sub(" ", body)))
    out = []
    for m in NUM_RE.finditer(body):
        v = int(re.sub(r"\D", "", m.group(1)))
        if 300 <= v <= 99999:
            out.append(v)
    return out


def parse_report(text, ref=None):
    """Hisobot kuni = 'Rd - dd.mm'. Chek yonidagi sana (08.10.) e'tiborga olinmay
    Rd chekdan keyingi alohida qatorda, shu qatorda yoki sarlavhada bo'lishi mumk
    ref = ref or today()
    ents = [(find_rd(line, ref), line_amounts(line)) for line in text.splitlines(
    mode, seen = "suffix", False
    for rd, amts in ents:
        if rd and not amts:
            mode = "suffix" if seen else "header"
            break
        if amts:
```

```
            seen = True
    slots, pending, cur = [], [], None
    for rd, amts in ents:
        if rd and amts:
            slots += [[rd, a] for a in amts]
            cur = rd
        elif rd:
            if mode == "suffix":
                for i in pending:
                    slots[i][0] = rd
                pending = []
            cur = rd
        elif amts:
            for a in amts:
                if mode == "header" and cur:
                    slots.append([cur, a])
                else:
                    slots.append([None, a])
                    pending.append(len(slots) - 1)
    fill = cur or ref
    return [(rd or fill, a) for rd, a in slots]


def report_checks(text, post, is_owner):
    """Egasi hisobotida Rd bo'lmasa: hamma qatorda bor sana (agar aniq bitta bo'l
    out = parse_report(text, post)
    if is_owner and out and not has_rd(text):
        lines = [(line_amounts(l), {d for d in parse_dates(l, post)}) for l in te
        L1 = [l for l in lines if l[0]]
        if L1 and all(l[1] for l in L1):
            common = set.intersection(*[l[1] for l in L1])
            if len(common) == 1:
                d = next(iter(common))
                return [(d, a) for _, a in out]
    return out


def split_tag(text):
    """Birinchi qator faqat akkaunt nomi bo'lsa ('Lazarus\n1950 2600'): -> (worke
    lines = text.strip().splitlines()
    if len(lines) >= 2:
        first = lines[0].strip().rstrip(":.-").strip()
        for w in workers(active_only=False, kinds=("worker", "self", "pool")):
            if first.lower() == w["name"].lower():
                rest = "\n".join(lines[1:])
                if is_report_like(rest):
```

```
                    return w["id"], rest
    return None, text


KIND_PAT = (r"штраф\w*|shtraf\w*|jarima\w*|бонус\w*|bonus\w*|премия\w*|прем\w*|pr
            r"|аванс\w*|avans\w*|avanc\w*")
KIND_RE = re.compile(KIND_PAT, re.I)


def has_kind(text):
    return bool(KIND_RE.search(text))


def kind_of(tok):
    if re.match(r"штраф|shtraf|jarima", tok, re.I):
        return "fine"
    if re.match(r"аванс|avans|avanc", tok, re.I):
        return "adv"
    return "bonus"


def parse_adjustments(text):
    """-> ([{kind, worker_id|None, usd}], toza_mi). Ismlar: hamma (Cashliy va Art
    ws = workers(kinds=("worker", "self", "partner"))
    names = sorted([w["name"] for w in ws], key=len, reverse=True)
    name_pat = "|".join(re.escape(n) for n in names) or "(?!x)x"
    by_name = {w["name"].lower(): w["id"] for w in ws}
    tok = re.compile(rf"(?P<kind>{KIND_PAT})|(?P<name>(?<!\w)(?:{name_pat})(?!\w)
    t2 = DATE_RE.sub(" ", text)
    kind = wid = None
    items, kinds, wids = [], set(), set()
    for m in tok.finditer(t2):
        if m.group("kind"):
            kind = kind_of(m.group("kind"))
            kinds.add(kind)
        elif m.group("name"):
            wid = by_name.get(m.group("name").lower())
            wids.add(wid)
        else:
            items.append({"kind": kind, "worker_id": wid, "usd": float(m.group("n
    for it in items:
        if it["kind"] is None and len(kinds) == 1:
            it["kind"] = next(iter(kinds))
        if it["worker_id"] is None and len(wids) == 1:
            it["worker_id"] = next(iter(wids))
    leftover = re.sub(r"[\s,;+$:/.\-—()!]+", "", tok.sub(" ", t2))
```

```
    return items, not leftover


# ──────────────────────────── hisobotlar ────────────────────────────
def report(rid):
    return q1("select * from reports where id=?", (rid,))


def report_amounts_by_msg(chat_id, msg_id):
    r = q1("select id from reports where chat_id=? and msg_id=?", (chat_id, msg_i
    return None if not r else [c["amount"] for c in q("select amount from checks


def amounts_of(rid):
    return [c["amount"] for c in q("select amount from checks where report_id=? o


def create_report(worker_id, chat_id, msg_id, sender_id, raw, checks, sender_name
    if msg_id is not None:
        for o in q("select id from reports where chat_id=? and msg_id=?", (chat_i
            delete_report(o["id"])
    cur = x("insert into reports(worker_id,chat_id,msg_id,sender_id,sender_name,r
            (worker_id, chat_id, msg_id, sender_id, sender_name, raw, dt.datetime
    rid = cur.lastrowid
    for rd, amt in checks:
        x("insert into checks(report_id,rd,amount,usd) values(?,?,?,?)", (rid, is
    return rid


def delete_report(rid):
    x("delete from checks where report_id=?", (rid,))
    x("delete from reports where id=?", (rid,))


def set_report_status(rid, st):
    x("update reports set status=? where id=?", (st, rid))


def is_pool(wid):
    w = worker(wid)
    return bool(w) and w["kind"] == "pool"


def report_needs(rid):
    r = report(rid)
    if r is None:
```

```
        return None
    if r["worker_id"] is None:
        return ("worker", None)
    if q1("select 1 from checks where report_id=? and rd is null", (rid,)):
        return ("date", None)
    row = q1("select amount from checks where report_id=? and usd is null limit 1
    if row:
        return ("price", row["amount"])
    if is_pool(r["worker_id"]) and q1("select 1 from checks where report_id=? and
        return ("op", None)
    return None


def pending_reports():
    return [r["id"] for r in q("select id from reports where status='pending'")]


def report_days(rid):
    return [r["rd"] for r in q("select distinct rd from checks where report_id=?


def supersede(rid):
    """Yangi hisobot eskisini to'liq ichiga olsa (>=2 chek) — eskisi hisobdan chi
    r = report(rid)
    new = amounts_of(rid)
    out = []
    for d in report_days(rid):
        for o in q("select distinct r.id from reports r join checks c on c.report
                   "where r.worker_id=? and r.status='done' and r.id!=? and c.rd=
            old = amounts_of(o["id"])
            pool = list(new)
            ok = len(old) >= 2
            for a in old:
                if a in pool:
                    pool.remove(a)
                else:
                    ok = False
                    break
            if ok:
                set_report_status(o["id"], "replaced")
                out.append({"old": o["id"], "amounts": old, "kind": "replace"})
            elif len(old) == 1 and old == new:
                out.append({"old": o["id"], "amounts": old, "kind": "dup"})
    return out
```

```
def set_pay_report(rid, wid):
    x("update checks set pay=? where report_id=? and pay is null", (wid, rid))


def set_pay_day(acct_id, day, wid):
    """Shu akkauntning shu kundagi hamma cheklari uchun oylik oluvchi (wid; None
    x("update checks set pay=? where rd=? and report_id in (select id from report
      (wid, day, acct_id))


def set_pay_check(cid, wid):
    x("update checks set pay=? where id=?", (wid, cid))


def unassigned_days(acct_id):
    return [r["rd"] for r in q("select distinct ch.rd from checks ch join reports
                               "where r.worker_id=? and r.status='done' and ch.pa
                               (acct_id,))]


def worker_days(wid, limit=10):
    return q("select ch.rd as rd, count(*) as n from checks ch join reports r on
             "where r.worker_id=? and r.status='done' and ch.rd is not null group


def day_checks(wid, day):
    return q("select ch.id, ch.amount, ch.usd, ch.skip, ch.pay from checks ch joi
             "where r.worker_id=? and r.status='done' and ch.rd=? order by ch.id"


def check_row(cid):
    return q1("select ch.*, r.worker_id as acct from checks ch join reports r on


def toggle_check(cid):
    x("update checks set skip=1-skip where id=?", (cid,))


def set_check_day(cid, day):
    x("update checks set rd=? where id=?", (day, cid))


def add_check(wid, day, amount):
    cur = x("insert into reports(worker_id,chat_id,sender_name,raw,status,created
            (wid, 0, "вручную", str(amount), "done", dt.datetime.now(TZ).isoforma
    x("insert into checks(report_id,rd,amount,usd) values(?,?,?,?)", (cur.lastrow
```

```
def edit_check_amount(cid, new):
    x("update checks set amount=?, usd=? where id=?", (new, price_for(new), cid))


def clear_day(wid, day):
    ids = [r["id"] for r in q("select distinct r.id from reports r join checks c
                              "where r.worker_id=? and c.rd=?", (wid, day))]
    for i in ids:
        x("delete from checks where report_id=? and rd=?", (i, day))
        if not q1("select 1 from checks where report_id=?", (i,)):
            x("delete from reports where id=?", (i,))
    return len(ids)


# ─────────────────────────── shtraf / bonus / avans ─────────────────────────
def add_adj(worker_id, kind, usd, day, raw, chat_id=None, msg_id=None, pending=0)
    return x("insert into adj(worker_id,kind,usd,day,raw,chat_id,msg_id,pending)
             (worker_id, kind, usd, day, raw, chat_id, msg_id, pending)).lastrowi


def adj(aid):
    return q1("select * from adj where id=?", (aid,))


def adj_set(aid, **kw):
    for k, v in kw.items():
        x(f"update adj set {k}=? where id=?", (v, aid))


def adj_toggle(aid):
    x("update adj set cancelled=1-cancelled where id=?", (aid,))


def adj_delete(aid):
    x("delete from adj where id=?", (aid,))


def drop_adj_msg(chat_id, msg_id):
    x("delete from adj where chat_id=? and msg_id=?", (chat_id, msg_id))


def pending_adjs():
    return [r["id"] for r in q("select id from adj where pending=1")]
```

```
def worker_adjs(wid, limit=15):
    return q("select * from adj where worker_id=? and pending=0 order by id desc


# ──────────────────────── o'tkazma va ulushlar ────────────────────────
def add_transfer(day, from_id, to_id, pct):
    return x("insert into transfers(day,from_id,to_id,pct) values(?,?,?,?)", (day


def transfer_toggle(tid):
    x("update transfers set cancelled=1-cancelled where id=?", (tid,))


def worker_transfers(wid, limit=10):
    return q("select * from transfers where from_id=? or to_id=? order by id desc


def shares_of(wid):
    return q("select * from shares where active=1 and (to_id=? or from_id=?) orde


def all_shares():
    return q("select * from shares where active=1 order by id")


def share_row(sid):
    return q1("select * from shares where id=?", (sid,))


def add_share(to_id, from_id, pct, base="zp", mode="deduct", from_day="2000-01-01
    return x("insert into shares(to_id,from_id,pct,base,mode,from_day) values(?,?
             (to_id, from_id, pct, base, mode, from_day)).lastrowid


def update_share(sid, **kw):
    for k, v in kw.items():
        x(f"update shares set {k}=? where id=?", (v, sid))


def del_share(sid):
    x("update shares set active=0 where id=?", (sid,))


# ──────────────────────── davrlar ────────────────────────
def period_range(key):
```

```
    t = today()
    if re.fullmatch(r"\d{4}-\d{2}-\d{2}", key):
        d = dt.date.fromisoformat(key)
        return d, d
    if re.fullmatch(r"\d{4}-\d{2}-\d{2}_\d{4}-\d{2}-\d{2}", key):
        a, b = key.split("_")
        return dt.date.fromisoformat(a), dt.date.fromisoformat(b)
    if key == "td":
        return t, t
    if key == "yd":
        y = t - dt.timedelta(days=1)
        return y, y
    if key == "tw":
        return week_start(t), week_start(t) + dt.timedelta(days=6)
    if key == "lw":
        mon = week_start(t) - dt.timedelta(days=7)
        return mon, mon + dt.timedelta(days=6)
    if key == "tm":
        first = t.replace(day=1)
        return first, (first + dt.timedelta(days=32)).replace(day=1) - dt.timedel
    if key == "lm":
        last = t.replace(day=1) - dt.timedelta(days=1)
        return last.replace(day=1), last
    return t, t


# ──────────────────────────── HISOB-KITOB ────────────────────────────
def _rows_empty(w):
    return {"w": w, "kind": w["kind"], "days": [], "n": 0, "raw": 0, "usd": 0.0,
            "fines": [], "bonuses": [], "advs": [], "fine": 0.0, "bonus": 0.0, "a
            "outs": [], "ins": [], "tr_out": 0.0, "tr_in": 0.0, "sin": [], "sout"
            "total": 0.0, "pf": 0.0, "pb": 0.0, "op_zp": OrderedDict(), "unassign


def calc_bundle(start, end):
    """Hamma narsani bir joyda hisoblaydi. -> {'rows': {wid: row}, 'order': [wid.
    s, e = iso(start), iso(end)
    ws = {w["id"]: w for w in workers(active_only=False)}
    rows = {wid: _rows_empty(w) for wid, w in ws.items()}
    partner = next((w for w in ws.values() if w["kind"] == "partner"), None)

    checks = q("select ch.id cid, ch.rd, ch.amount, ch.usd, ch.skip, ch.pay, r.wo
               "from checks ch join reports r on r.id=ch.report_id "
               "where r.status='done' and ch.rd between ? and ? order by ch.rd, c
    pay_days = {}          # wid -> OrderedDict(day -> [(amount, usd, src)])
    acc_days = {}          # acct -> OrderedDict(day -> {'items': [], 'ops': set(
```

```
Kday, Rday, Nday = {}, {}, {}
for c in checks:
    if c["skip"]:
        continue
    usd = c["usd"] if c["usd"] is not None else 0.0
    a = ws.get(c["acct"])
    if a is None:
        continue
    payee = c["pay"] or (None if a["kind"] == "pool" else c["acct"])
    d = c["rd"]
    Kday[d] = Kday.get(d, 0.0) + usd
    Rday[d] = Rday.get(d, 0) + c["amount"]
    Nday[d] = Nday.get(d, 0) + 1
    ad = acc_days.setdefault(c["acct"], OrderedDict()).setdefault(d, {"items"
    ad["items"].append((c["amount"], usd, None))
    pn = ws[payee]["name"] if payee and payee in ws else None
    if pn and pn not in ad["ops"]:
        ad["ops"].append(pn)
    if payee is None:
        rows[c["acct"]]["unassigned"] += 1
        continue
    if payee in ws:
        pay_days.setdefault(payee, OrderedDict()).setdefault(d, []).append(
            (c["amount"], usd, a["name"] if payee != c["acct"] else None))

# 1) oylik oladiganlar: kunlar va zp
base_zp = {}           # (wid, day) -> zp (o'tkazma/ulush uchun)
base_usd = {}
for wid, w in ws.items():
    r = rows[wid]
    if w["kind"] in PAYABLE:
        zp = 0.0
        for d, items in pay_days.get(wid, {}).items():
            usd = sum(u for _, u, _ in items)
            p = percent_on(wid, d)
            z = usd * p / 100
            r["days"].append({"day": d, "items": items, "raw": sum(a for a, _
            base_zp[(wid, d)] = z
            base_usd[(wid, d)] = usd
            zp += z
            bysrc = {}
            for a_, u_, src_ in items:
                if src_:
                    bysrc[src_] = bysrc.get(src_, 0.0) + u_
            for src_, u_ in bysrc.items():            # akkaunt egasiga: "kim
                sid_ = next((i for i, w_ in ws.items() if w_["name"] == src_)
```

```
                if sid_:
                    rows[sid_]["op_zp"].setdefault(w["name"], []).append({"da
        r["n"] = sum(len(i) for i in pay_days.get(wid, {}).values())
        r["raw"] = sum(d["raw"] for d in r["days"])
        r["usd"] = sum(d["usd"] for d in r["days"])
        r["zp"] = zp
    elif w["kind"] == "pool":
        for d, ad in acc_days.get(wid, {}).items():
            usd = sum(u for _, u, _ in ad["items"])
            r["days"].append({"day": d, "items": ad["items"], "raw": sum(a fo
                              "pct": 0.0, "zp": 0.0, "ops": ad["ops"]})
        r["n"] = sum(len(d["items"]) for d in r["days"])
        r["raw"] = sum(d["raw"] for d in r["days"])
        r["usd"] = sum(d["usd"] for d in r["days"])
# 2) hamkor (Artur): kassadan foiz
if partner:
    r = rows[partner["id"]]
    for d in sorted(Kday):
        p = percent_on(partner["id"], d)
        z = Kday[d] * p / 100
        r["days"].append({"day": d, "items": [], "raw": Rday[d], "usd": Kday[
        base_zp[(partner["id"], d)] = z
        r["zp"] += z
    r["n"] = sum(Nday.values())
    r["raw"] = sum(Rday.values())
    r["usd"] = sum(Kday.values())

# 3) shtraf/bonus/avans
for a in q("select * from adj where cancelled=0 and pending=0 and day between
    r = rows.get(a["worker_id"])
    if r is None:
        continue
    key = {"fine": "fines", "bonus": "bonuses", "adv": "advs"}.get(a["kind"])
    if key:
        r[key].append((a["usd"], a["day"]))
for r in rows.values():
    r["fine"] = sum(v for v, _ in r["fines"])
    r["bonus"] = sum(v for v, _ in r["bonuses"])
    r["adv"] = sum(v for v, _ in r["advs"])

# 4) o'tkazmalar (shu kundagi zp ning foizi)
for t in q("select * from transfers where cancelled=0 and day between ? and ?
    if t["from_id"] not in rows or t["to_id"] not in rows:
        continue
    amt = r2(base_zp.get((t["from_id"], t["day"]), 0.0) * t["pct"] / 100)
    rows[t["from_id"]]["outs"].append({"id": t["id"], "day": t["day"], "other
```

```
    rows[t["to_id"]]["ins"].append({"id": t["id"], "day": t["day"], "other":
for r in rows.values():
    r["tr_out"] = sum(o["amount"] for o in r["outs"])
    r["tr_in"] = sum(i["amount"] for i in r["ins"])

# 5) ulushlar (masalan Bayzed <- Baxa 5%)
owner_paid = 0.0
for sh in q("select * from shares where active=1"):
    if sh["from_id"] not in rows or sh["to_id"] not in rows:
        continue
    for d, items in pay_days.get(sh["from_id"], {}).items():
        if d < sh["from_day"] or not (s <= d <= e):
            continue
        base = base_zp.get((sh["from_id"], d), 0.0) if sh["base"] == "zp" els
        amt = r2(base * sh["pct"] / 100)
        if amt == 0:
            continue
        rows[sh["to_id"]]["sin"].append({"day": d, "amount": amt, "other": ws
        if sh["mode"] == "deduct":
            rows[sh["from_id"]]["sout"].append({"day": d, "amount": amt, "oth
        else:
            owner_paid += amt
for r in rows.values():
    r["sin_sum"] = sum(i["amount"] for i in r["sin"])
    r["sout_sum"] = sum(i["amount"] for i in r["sout"])

# 6) itogo
pf = sum(r["fine"] for r in rows.values() if r["kind"] in PAYABLE)
pb = sum(r["bonus"] for r in rows.values() if r["kind"] in PAYABLE)
for r in rows.values():
    r["zp"] = r2(r["zp"])
    r["usd"] = r2(r["usd"])
    if r["kind"] in PAYABLE:
        r["total"] = r2(r["zp"] - r["fine"] + r["bonus"] - r["adv"] - r["tr_o
    elif r["kind"] == "partner":
        r["pf"], r["pb"] = r2(pf), r2(pb)
        r["total"] = r2(r["zp"] - r["fine"] + r["bonus"] - r["adv"] - r["tr_o
K, RAW, N = sum(Kday.values()), sum(Rday.values()), sum(Nday.values())
# 7) ega cho'ntagi
owner_half = sum(Kday[d] * (100 - (percent_on(partner["id"], d) if partner el
zps = sum(r["zp"] for r in rows.values() if r["kind"] == "worker")
me = next((r for r in rows.values() if r["kind"] == "self"), None)
out_self = me["tr_out"] if me else 0.0
in_self = me["tr_in"] if me else 0.0
owner_income = r2(owner_half - zps - owner_paid - out_self + in_self)
owner = {"K": r2(K), "half": r2(owner_half), "zps": r2(zps), "owner_paid": r2
```

```
                "out_self": r2(out_self), "in_self": r2(in_self), "income": owner_in
                "artur": rows[partner["id"]]["total"] if partner else 0.0,
                "artur_share": rows[partner["id"]]["zp"] if partner else 0.0}
    accounts = []
    for wid, w in ws.items():
        if w["kind"] in ("worker", "self", "pool"):
            ad = acc_days.get(wid, {})
            raw = sum(sum(a for a, _, _ in d["items"]) for d in ad.values())
            n = sum(len(d["items"]) for d in ad.values())
            usd = sum(sum(u for _, u, _ in d["items"]) for d in ad.values())
            accounts.append({"w": w, "raw": raw, "n": n, "usd": usd})
    order = [w["id"] for w in workers(active_only=False)]
    return {"rows": rows, "order": order, "K": r2(K), "raw": RAW, "n": N, "owner"


def has_data(r):
    if r["kind"] == "partner":
        return r["usd"] > 0 or bool(r["fines"] or r["bonuses"] or r["advs"] or r[
    return bool(r["n"] or r["fines"] or r["bonuses"] or r["advs"] or r["outs"] or


def calc(start, end, only=None):
    b = calc_bundle(start, end)
    ids = [only] if only else b["order"]
    out = []
    for wid in ids:
        r = b["rows"].get(wid)
        if r is None:
            continue
        if not only and not r["w"]["active"] and not has_data(r):
            continue
        out.append(r)
    return out


def day_base(wid, day):
    """O'tkazma uchun: shu kundagi zp (partner uchun - ulushi)."""
    d = dt.date.fromisoformat(day) if isinstance(day, str) else day
    r = calc_bundle(d, d)["rows"].get(wid)
    return r["zp"] if r else 0.0


# ─────────────────────────── matnlar (ruscha) ────────────────────────────
def money(v):
    v = r2(v)
    return str(int(v)) if abs(v - int(v)) < 1e-9 else f"{v:.2f}"
```

```
def n3(v):
    return f"{int(v):,}"


def dshort(d):
    return dt.date.fromisoformat(d).strftime("%d.%m")


def label(s, e):
    return dshort(iso(s)) if s == e else f"{dshort(iso(s))}–{dshort(iso(e))}"


def chk_word(n):
    if n % 10 == 1 and n % 100 != 11:
        return "чек"
    if 2 <= n % 10 <= 4 and not 12 <= n % 100 <= 14:
        return "чека"
    return "чеков"


def adj_list(items):
    v = money(sum(a for a, _ in items))
    return f"{v}$ (" + "+".join(money(a) for a, _ in items) + ")" if len(items) >


def pay_lines(r):
    """Зп — faqat summa. Qolganlari FAQAT 0 dan katta bo'lsa. Itogo faqat bo'lsa.
    lines = [f"Зп: {money(r['zp'])}$"]
    expr, extra = money(r["zp"]), False

    def add(line, sign, val):
        nonlocal expr, extra
        lines.append(line)
        expr += f"{sign}{money(val)}"
        extra = True
    if r["fine"] > 0:
        add(f"Штраф: {adj_list(r['fines'])}", "-", r["fine"])
    if r["bonus"] > 0:
        add(f"Бонус: {adj_list(r['bonuses'])}", "+", r["bonus"])
    if r["adv"] > 0:
        add(f"Аванс: {adj_list(r['advs'])}", "-", r["adv"])
    for o in r["outs"]:
        if o["amount"] > 0:
            add(f"Перенос → {o['other']} ({dshort(o['day'])}): −{money(o['amount'
```

```
    for i in r["ins"]:
        if i["amount"] > 0:
            add(f"Перенос от {i['other']} ({dshort(i['day'])}): +{money(i['amount
    for i in r["sin"]:
        add(f"Доля {i['pct']:g}% с {i['other']} ({dshort(i['day'])}): +{money(i['
    for i in r["sout"]:
        add(f"Отдаёт {i['pct']:g}% → {i['other']} ({dshort(i['day'])}): −{money(i
    if extra:
        lines.append(f"Итого: {expr}={money(r['total'])}$")
    return lines


def partner_lines(r):
    lines = [f"Доля: {money(r['usd'])}$ × {money(r['days'][0]['pct']) if r['days'
    expr = money(r["zp"])
    if r["pf"] > 0:
        lines.append(f"Штрафы с обработчиков: +{money(r['pf'])}$")
        expr += f"+{money(r['pf'])}"
    if r["pb"] > 0:
        lines.append(f"Бонусы обработчикам: −{money(r['pb'])}$")
        expr += f"-{money(r['pb'])}"
    if r["fine"] > 0:
        lines.append(f"Штраф: {adj_list(r['fines'])}")
        expr += f"-{money(r['fine'])}"
    if r["bonus"] > 0:
        lines.append(f"Бонус: {adj_list(r['bonuses'])}")
        expr += f"+{money(r['bonus'])}"
    if r["adv"] > 0:
        lines.append(f"Аванс: {adj_list(r['advs'])}")
        expr += f"-{money(r['adv'])}"
    for o in r["outs"]:
        lines.append(f"Перенос → {o['other']} ({dshort(o['day'])}): −{money(o['am
        expr += f"-{money(o['amount'])}"
    for i in r["ins"]:
        lines.append(f"Перенос от {i['other']} ({dshort(i['day'])}): +{money(i['a
        expr += f"+{money(i['amount'])}"
    lines.append(f"Итого: {expr}={money(r['total'])}$")
    return lines


def zp_text(r, lab):
    """Ishchining o'ziga boradigan / toza oylik matni (shtraf-bonus hisoblangan h
    if r["kind"] == "partner":
        return "\n".join([f"{r['w']['name']} · {lab}"] + partner_lines(r))
    return "\n".join([f"{r['w']['name']} · {lab}"] + pay_lines(r))
```

```
def brief(r, lab=None, updated=False):
    """Qisqa xabar: sana, kassa, rd soni, kassa $."""
    w = r["w"]
    L = []
    if updated:
        L.append("♻️ Отчёт обновлён (старый заменён)")
    L.append(w["name"] + (" (общий аккаунт)" if w["kind"] == "pool" else ""))
    days = r["days"]
    if len(days) == 1:
        d = days[0]
        L += [f"Дата: {dshort(d['day'])}", f"Касса: {n3(d['raw'])}", f"Кол-во рд:
        if w["kind"] == "pool" and d.get("ops"):
            L.append("Работал: " + ", ".join(d["ops"]))
    else:
        if lab:
            L[-1] += f" · {lab}"
        for d in days:
            ops = f" · {', '.join(d['ops'])}" if w["kind"] == "pool" and d.get("o
            L.append(f"{dshort(d['day'])} · касса {n3(d['raw'])} · рд {len(d['ite
        L += ["", f"Всего: касса {n3(r['raw'])} · кол-во рд {r['n']} · {money(r['
    if r["unassigned"]:
        L.append(f"⚠️ {r['unassigned']} чек(ов) без исполнителя — выберите, кто р
    return "\n".join(L)


def explain(r, lab, owner=False):
    """Razbor: faqat admin va egaga (tekshirish uchun hamma narsa ko'rinadi)."""
    w = r["w"]
    L = [f"🔎 Разбор: {w['name']} · {lab}"]
    if w["kind"] == "partner":
        for d in r["days"]:
            L.append(f"{dshort(d['day'])}: касса {money(d['usd'])}$ × {money(d['p
        L.append("")
        L += partner_lines(r)
        return "\n".join(L)
    if not r["days"]:
        L.append("Чеков за период нет.")
    for d in r["days"]:
        L.append("")
        ops = f" · работал: {', '.join(d['ops'])}" if w["kind"] == "pool" and d.g
        L.append(f"📅 {dshort(d['day'])} — кол-во рд: {len(d['items'])}{ops}")
        cnt = OrderedDict()
        for a, u, src in d["items"]:
            cnt[(a, u, src)] = cnt.get((a, u, src), 0) + 1
        for (a, u, src), k in cnt.items():
```

```
            tag = f" [{src}]" if src else ""
            if u == 0:
                L.append(f"{a}{tag} = ❓ нет цены")
            elif k > 1:
                L.append(f"{a}{tag} = {money(u)}$ × {k} = {money(u * k)}$")
            else:
                L.append(f"{a}{tag} = {money(u)}$")
        L.append("Касса: " + " + ".join(str(a) for a, _, _ in d["items"]) + f" =
        L.append(f"По прайсу: {money(d['usd'])}$")
        if w["kind"] != "pool":
            L.append(f"Зп: {money(d['usd'])}$ × {money(d['pct'])}% = {money(d['zp
    L.append("")
    if r["op_zp"]:
        L.append("Кто получил зп с этого аккаунта:")
        for opn, lst in r["op_zp"].items():
            for it in lst:
                L.append(f"  {opn} ({dshort(it['day'])}): {money(it['usd'])}$ × {
        L.append("")
    if w["kind"] == "pool":
        L.append("Общий аккаунт: зп получает тот, кто работал (по своему проценту
    else:
        L += pay_lines(r)
    if owner and w["kind"] == "worker" and r["n"]:
        L.append("")
        L.append(f"💼 Ваша часть: {money(r['usd'])}$ × 50% − зп {money(r['zp'])}$
    return "\n".join(L)


def visible_rows(res, owner):
    return [r for r in res if has_data(r) and (owner or r["kind"] in ("worker", "


def overall_text(b, lab, owner=False):
    """Umumiy hisobot. Ega ko'rinishi: Cashliy qatori + Artur + 'Ваш доход'. Admi
    rows = [b["rows"][i] for i in b["order"] if has_data(b["rows"][i])]
    L = [f"📊 Общий отчёт ({lab})", ""]
    tn = traw = 0
    tusd = 0.0
    for r in rows:
        k = r["kind"]
        if k == "partner":
            continue
        if k == "self" and not owner:
            continue
        if k == "pool":
            L.append(f"{r['w']['name']} (общий аккаунт): {r['n']} {chk_word(r['n'
```

```
            if r["unassigned"]:
                L.append(f"   ⚠️ {r['unassigned']} чек(ов) без исполнителя")
            continue
        L.append(f"{r['w']['name']}: {r['n']} {chk_word(r['n'])}, касса {n3(r['ra
    for a in b["accounts"]:
        if a["w"]["kind"] == "self" and not owner:
            continue
        tn += a["n"]
        traw += a["raw"]
        tusd += a["usd"]
    L += ["", f"Всего чеков: {tn}", f"Касса: {n3(traw)}", f"Касса ($): {money(tus
    if owner:
        L += ["", "💼 Ваши итоги (видно только вам):"] + owner_lines(b)
    return "\n".join(L)


def owner_lines(b):
    o = b["owner"]
    pr = b["rows"][next((i for i in b["order"] if b["rows"][i]["kind"] == "partne
        b["rows"][i]["kind"] == "partner" for i in b["order"]) else None
    p = pr["days"][0]["pct"] if pr and pr["days"] else 50
    L = [f"Общая касса: {money(o['K'])}$",
         f"Артур (партнёр): {money(o['K'])} × {money(p)}% = {money(o['artur_share
         f"Ваша половина: {money(o['half'])}$",
         f"Зарплаты обработчиков: {money(o['zps'])}$"]
    if o["owner_paid"]:
        L.append(f"Доли, которые платите вы: {money(o['owner_paid'])}$")
    if o["out_self"]:
        L.append(f"Ваши переносы другим: −{money(o['out_self'])}$")
    if o["in_self"]:
        L.append(f"Переносы вам: +{money(o['in_self'])}$")
    L.append(f"Ваш доход: {money(o['half'])} − {money(o['zps'])}"
             + (f" − {money(o['owner_paid'])}" if o["owner_paid"] else "")
             + (f" − {money(o['out_self'])}" if o["out_self"] else "")
             + (f" + {money(o['in_self'])}" if o["in_self"] else "")
             + f" = {money(o['income'])}$")
    me = next((b["rows"][i] for i in b["order"] if b["rows"][i]["kind"] == "self"
    if me and me["n"]:
        L.append(f"(в нём ваша зп Cashliy: {money(me['zp'])}$)")
    if pr and has_data(pr):
        L.append(f"Артур итого: {money(o['artur'])}$")
    return L


def rating_text(b, lab):
    rows = sorted([a for a in b["accounts"] if a["raw"] > 0], key=lambda a: -a["r
```

```
    if not rows:
        return f"🏆 Рейтинг кассы ({lab}): данных нет"
    medal = ["🥇", "🥈", "🥉"]
    L = [f"🏆 Рейтинг кассы ({lab})", ""]
    for i, a in enumerate(rows):
        m = medal[i] if i < 3 else (f"{i + 1}\ufe0f\u20e3" if i < 9 else f"{i + 1
        L.append(f"{m} 🟢 {a['w']['name']} — {n3(a['raw'])}")
    return "\n".join(L)


def chunks(text, limit=3800):
    out, cur = [], ""
    for line in text.split("\n"):
        if len(cur) + len(line) + 1 > limit and cur:
            out.append(cur)
            cur = ""
        cur += line + "\n"
    if cur.strip():
        out.append(cur.rstrip("\n"))
    return out or [""]
```